

50 Frequently Asked SQL Interview Questions

Answers, explanations, example queries and flow diagrams (2026)

ABOUT THIS GUIDE

This guide covers 50 of the most frequently asked SQL interview questions, drawn from several 2026 interview-prep guides. Every question has a plain-language explanation, an example query with its real output, and a flow diagram wherever a picture makes the idea easier to follow.

All example queries were run against the sample database described below, using SQLite, and the result tables show the actual output. Where syntax differs between PostgreSQL, MySQL, SQL Server and Oracle, the difference is noted. A few database-specific snippets (for example PERCENTILE_CONT and SQL Server pagination) are shown for reference and were not executed.

HOW TO USE IT

- Read the **Answer** first and try to say it in your own words.
- Study the **Flow** diagram to see the steps the query takes.
- Run the **Example** on your own data and change one thing at a time.
- Read the **Interview tip** for the follow-up question the interviewer is likely to ask.

PATTERN CHEAT SHEET

Most live-coding problems are one of these patterns. Spot the pattern in the wording, and the solution follows.

Pattern	How it sounds	Typical solution	Q
Nth highest value	"Second highest salary"	DENSE_RANK() = N	19-21
Top-N per group	"Top 3 earners in each department"	ROW_NUMBER() OVER (PARTITION BY ...), filter in an outer query	22
Finding duplicates	"Find duplicate emails"	GROUP BY + HAVING COUNT(*) > 1	23
Deleting duplicates	"Keep the latest or lowest-id row"	ROW_NUMBER() partitioned by the duplicate key; delete rn > 1	24
Running total / moving average	"Cumulative revenue", "7-day average"	SUM() or AVG() OVER (ORDER BY ... ROWS BETWEEN ...)	25-26
Gaps and islands	"Longest login streak", "missing IDs"	Value minus ROW_NUMBER() to tag runs; LEAD() for gaps	27-29
Row-over-row change	"Month-over-month growth"	LAG() / LEAD()	30
Cohorts and sessions	"Retention", "group events into sessions"	First-month CTE; LAG() gap flag + running SUM()	32-33

SAMPLE DATABASE

Most examples use two tables. One employee (Zoya) has no department, and one department (Legal) has no employees, which makes join behaviour easy to see. Smaller tables for a specific question are shown right next to that question.

employees

emp_id	name	dept_id	salary	manager_id
1	Asha	1	90000	NULL
2	Ravi	1	80000	1
3	Meera	1	80000	1
4	Karan	1	85000	2
5	Neha	2	75000	NULL
6	Arjun	2	65000	5
7	Divya	2	65000	5
8	Isha	2	60000	5
9	Vikram	3	55000	NULL
10	Sana	3	50000	9
11	Zoya	NULL	45000	9

departments

dept_id	dept_name
1	Engineering
2	Sales
3	HR
4	Legal

QUESTION INDEX

Q1	What are the main types of SQL commands (DDL, DML, DCL, TCL)?
Q2	What is the difference between a primary key and a foreign key?
Q3	What is a composite primary key?
Q4	What types of constraints does SQL support?
Q5	What is the difference between DELETE, TRUNCATE and DROP?
Q6	What is the difference between WHERE and HAVING?
Q7	What is the difference between UNION and UNION ALL?
Q8	How does SQL treat NULL, and how do you compare against it?
Q9	What is the logical order of execution of a SQL query?
Q10	What are aggregate functions, and how do they work with GROUP BY?
Q11	How do LIMIT / TOP and ORDER BY work together for pagination?
Q12	What is normalization, and when would you denormalize?
Q13	Explain INNER, LEFT, RIGHT and FULL joins.
Q14	What is a CROSS join, and when is it useful?
Q15	What is a self join? Find employees who earn more than their manager.
Q16	How do you find rows in one table that have no match in another?
Q17	What is the difference between a JOIN and a subquery?
Q18	What is the difference between a correlated and a nested subquery?
Q19	Find the second highest salary.

Q20	Find the Nth highest salary.
Q21	Find the second highest salary in each department.
Q22	Find the top 3 earners in each department.
Q23	Find duplicate rows (for example, duplicate emails).
Q24	Delete duplicate rows while keeping one.
Q25	Calculate a running total of sales by date.
Q26	Calculate a 7-day moving average.
Q27	Find users who logged in on 3 or more consecutive days.
Q28	Find the longest login streak per user.
Q29	Find missing IDs or missing dates in a sequence.
Q30	Calculate month-over-month change using LAG().
Q31	Calculate the median salary.
Q32	Calculate customer retention by cohort.
Q33	Group events into sessions (sessionization).
Q34	Pivot rows into columns, and unpivot columns into rows.
Q35	What is a window function, and how does it differ from GROUP BY?
Q36	What is the difference between RANK, DENSE_RANK and ROW_NUMBER?
Q37	What does PARTITION BY do?
Q38	How do window frame clauses (ROWS BETWEEN) work?
Q39	What is a CTE, and when would you use one over a subquery?
Q40	How do you rewrite a correlated subquery as a JOIN?
Q41	What is an index, and how does it speed up queries?
Q42	When can an index hurt performance?
Q43	Why should you avoid wrapping an indexed column in a function in WHERE?
Q44	How do you read an execution plan (EXPLAIN)?
Q45	How would you optimize a slow query?
Q46	What are the ACID properties?
Q47	What are transaction isolation levels, and what is a dirty or phantom read?
Q48	What is a deadlock, and how do you prevent it?
Q49	How do you defend against SQL injection?
Q50	How would you design a schema that scales?

1. Fundamentals

Warm-up questions that appear in almost every interview. Short, precise answers score best here.

Q1. What are the main types of SQL commands (DDL, DML, DCL, TCL)?

ANSWER

SQL commands are grouped by what they do. Interviewers want to see that you know which group a command belongs to, because the group decides things such as whether the change can be rolled back and who is allowed to run it.

Group	Stands for	Purpose	Main commands
DDL	Data Definition Language	Define or change the structure of objects	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Add, change or remove the data inside tables	INSERT, UPDATE, DELETE, MERGE
DQL	Data Query Language	Read data (many guides count it inside DML)	SELECT
DCL	Data Control Language	Control who may do what	GRANT, REVOKE
TCL	Transaction Control Language	Control transactions	COMMIT, ROLLBACK, SAVEPOINT

Interview tip: A common follow-up is "Is TRUNCATE DDL or DML?" Answer: DDL, because it works on the table as a whole rather than row by row.

Q2. What is the difference between a primary key and a foreign key?

ANSWER

A **primary key** uniquely identifies each row in its own table. A **foreign key** is a column that points to the primary key of another table, so the database can guarantee that the link is valid (referential integrity).

	Primary key	Foreign key
Purpose	Identify a row	Link rows in two tables
Duplicates allowed	No	Yes (many employees can share one department)
NULL allowed	No	Yes, unless the column is also NOT NULL
How many per table	Exactly one (it may span several columns)	As many as needed

EXAMPLE

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);
```

With this foreign key, inserting an employee with `dept_id = 99` fails if no such department exists, and deleting a department that still has employees is blocked (unless you add `ON DELETE CASCADE` or `SET NULL`).

Q3. What is a composite primary key?

ANSWER

A composite primary key uses two or more columns together to identify a row. Each column on its own may repeat, but the combination must be unique and none of the columns may be NULL. It is typical for link tables in many-to-many relationships.

EXAMPLE

```
CREATE TABLE order_items (  
    order_id INT,  
    product_id INT,  
    quantity INT NOT NULL,  
    PRIMARY KEY (order_id, product_id)  
);
```

Order 10 can appear many times, and product 7 can appear many times, but the pair (10, 7) can appear only once. That stops the same product being added twice to the same order.

Q4. What types of constraints does SQL support?

ANSWER

Constraints are rules the database enforces so that bad data cannot get in. Enforcing them in the database is safer than relying on every application to behave.

Constraint	What it enforces
NOT NULL	The column must always have a value
UNIQUE	No two rows may have the same value (NULLs are handled differently by each database)
PRIMARY KEY	NOT NULL + UNIQUE, and identifies the row
FOREIGN KEY	The value must exist in the referenced table
CHECK	The value must satisfy a condition, e.g. salary > 0
DEFAULT	Supplies a value when none is given

EXAMPLE

```
CREATE TABLE staff (  
  staff_id INT PRIMARY KEY,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  salary INT CHECK (salary > 0),  
  status VARCHAR(10) DEFAULT 'active'  
);
```

Q5. What is the difference between DELETE, TRUNCATE and DROP?

ANSWER

	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
What it removes	Chosen rows (or all rows)	All rows	The whole table, including its structure
WHERE clause	Yes	No	No
Table structure kept	Yes	Yes	No
Speed on big tables	Slower (row by row, logged)	Much faster (deallocates pages)	Fast
Auto-increment counter	Keeps counting	Usually resets	Gone with the table
Triggers fire	Yes (row triggers)	No	No

Rollback depends on the database. In MySQL and Oracle, TRUNCATE and DROP commit implicitly, so they cannot be undone. PostgreSQL and SQL Server allow them inside a transaction, so they can be rolled back until you commit. Always say "it depends on the database" in an interview and name the ones you know.

EXAMPLE

```
DELETE FROM staff WHERE status = 'inactive'; -- some rows  
TRUNCATE TABLE staff; -- all rows, keep table  
DROP TABLE staff; -- table is gone
```

Q6. What is the difference between WHERE and HAVING?

ANSWER

WHERE filters individual rows before they are grouped. **HAVING** filters groups after GROUP BY, so it can use aggregate functions such as AVG() or COUNT(). The reason is the order of execution (see Q9): WHERE runs before grouping, HAVING runs after.

EXAMPLE

Departments whose average salary is above 70,000, counting only employees who earn more than 50,000:

```
SELECT dept_id, AVG(salary) AS avg_salary  
FROM employees  
WHERE salary > 50000 -- row filter, before grouping  
GROUP BY dept_id  
HAVING AVG(salary) > 70000; -- group filter, after grouping
```

Result

dept_id	avg_salary
1	83750

Interview tip: A good one-liner: "WHERE filters rows, HAVING filters groups." If a condition does not need an aggregate, put it in WHERE, because it removes rows earlier and is usually faster.

Q7. What is the difference between UNION and UNION ALL?

ANSWER

Both stack the results of two queries that have the same number of columns with compatible types. **UNION** removes duplicate rows, which requires a sort or hash step. **UNION ALL** keeps every row and is faster, so use it unless you really need de-duplication.

EXAMPLE

Query A returns everyone in Engineering. Query B returns everyone earning 85,000 or more. Asha and Karan are in both.

```
SELECT name FROM employees WHERE dept_id = 1
UNION
SELECT name FROM employees WHERE salary >= 85000
ORDER BY name;
```

UNION: 4 rows (duplicates removed)

name
Asha
Karan
Meera
Ravi

```
SELECT name FROM employees WHERE dept_id = 1
UNION ALL
SELECT name FROM employees WHERE salary >= 85000
ORDER BY name;
```

UNION ALL: 6 rows (Asha and Karan appear twice)

name
Asha
Asha
Karan
Karan
Meera
Ravi

Q8. How does SQL treat NULL, and how do you compare against it?

ANSWER

NULL means "unknown or missing". It is not zero and not an empty string. Any comparison with NULL, including NULL = NULL, gives UNKNOWN, and a WHERE clause only keeps rows where the condition is TRUE. So you must use **IS NULL** and **IS NOT NULL**.

Expression	Result
NULL = NULL	UNKNOWN (row is not returned)
5 > NULL	UNKNOWN
NULL AND TRUE	UNKNOWN
NULL OR TRUE	TRUE
NOT NULL	UNKNOWN

EXAMPLE

Sample data: employees (used throughout this guide). Zoya has no department.

emp_id	name	dept_id	salary	manager_id
1	Asha	1	90000	NULL
2	Ravi	1	80000	1
3	Meera	1	80000	1
4	Karan	1	85000	2
5	Neha	2	75000	NULL
6	Arjun	2	65000	5
7	Divya	2	65000	5
8	Isha	2	60000	5
9	Vikram	3	55000	NULL
10	Sana	3	50000	9
11	Zoya	NULL	45000	9

```
SELECT COUNT(*) AS all_rows, COUNT(dept_id) AS rows_with_dept FROM employees;
```

COUNT() counts rows; COUNT(column) skips NULLs*

all_rows	rows_with_dept
11	10

```
SELECT name FROM employees WHERE dept_id = NULL;
```

= NULL never matches: 0 rows

name
(0 rows)

```
SELECT name FROM employees WHERE dept_id IS NULL;
```

IS NULL works

name
Zoya

The NOT IN trap. If a subquery returns even one NULL, **NOT IN** returns no rows at all, because every comparison becomes UNKNOWN. Here employees.dept_id contains a NULL (Zoya), so this query wrongly returns nothing, although Legal has no employees:

```
SELECT dept_name FROM departments
WHERE dept_id NOT IN (SELECT dept_id FROM employees);
```

Returns 0 rows (wrong)

dept_name

(0 rows)

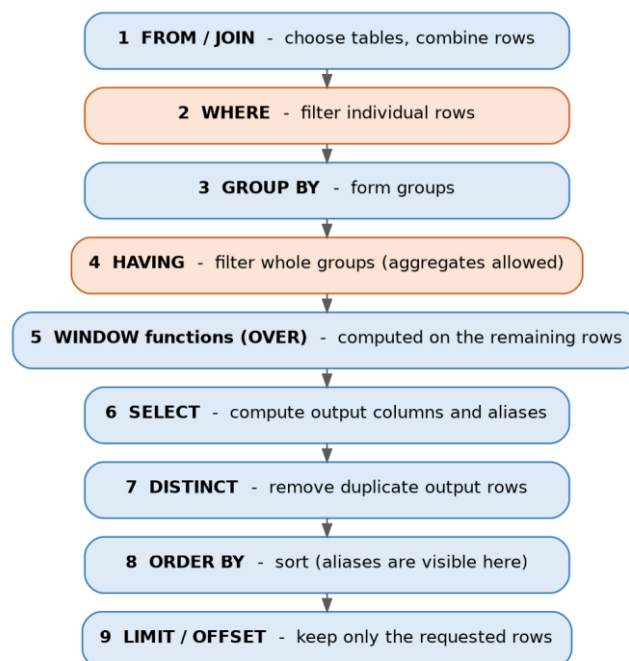
Use NOT EXISTS instead (see Q16). Other useful tools: **COALESCE**(col, 0) replaces NULL with a default, and **NULLIF**(a, b) returns NULL when the two values are equal (handy to avoid divide-by-zero).

Q9. What is the logical order of execution of a SQL query?

ANSWER

SQL is executed in a different order from the way you write it. This explains most "why does my query error?" puzzles: you cannot use a column alias in WHERE because SELECT has not run yet, you cannot use an aggregate in WHERE because grouping has not happened yet, and you cannot filter on a window function in WHERE for the same reason.

FLOW



The orange steps are the two filters. WHERE sees rows, HAVING sees groups.

EXAMPLE

```

SELECT dept_id, AVG(salary) AS avg_sal  -- 6. alias avg_sal is created here
FROM employees                          -- 1.
WHERE salary > 50000                    -- 2. cannot use avg_sal here
GROUP BY dept_id                       -- 3.
HAVING AVG(salary) > 70000              -- 4. aggregate allowed here
ORDER BY avg_sal DESC                   -- 8. alias allowed here
LIMIT 5;                               -- 9.
```

Interview tip: Mention that this is the logical order. The optimizer may physically run steps in a different order as long as the result is the same.

Q10. What are aggregate functions, and how do they work with GROUP BY?

ANSWER

Aggregate functions collapse many rows into one value: COUNT, SUM, AVG, MIN, MAX. Without GROUP BY they summarize the whole table. With GROUP BY they produce one result per group.

- Aggregates **ignore NULLs**, except **COUNT(*)**, which counts rows.
- Every column in SELECT that is not inside an aggregate must appear in GROUP BY.
- **COUNT(DISTINCT col)** counts unique values.

EXAMPLE

```
SELECT dept_id,
       COUNT(*)           AS headcount,
       ROUND(AVG(salary)) AS avg_salary,
       MAX(salary)        AS top_salary
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
ORDER BY dept_id;
```

Result

dept_id	headcount	avg_salary	top_salary
1	4	83750	90000
2	4	66250	75000
3	2	52500	55000

Q11. How do LIMIT / TOP and ORDER BY work together for pagination?

ANSWER

Without ORDER BY, the database may return rows in any order, so LIMIT on its own is not reliable. To page through results, sort first, then skip rows (OFFSET) and take a fixed number (LIMIT). Always add a unique tie-breaker column to the ORDER BY so that pages do not overlap.

Database	Syntax for "rows 4 to 6"
PostgreSQL, MySQL, SQLite	ORDER BY ... LIMIT 3 OFFSET 3
SQL Server	ORDER BY ... OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY (or SELECT TOP 3 for the first rows)
Oracle 12c and later	ORDER BY ... OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY

EXAMPLE

Page 2 of the highest paid employees, 3 per page:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC, emp_id  -- emp_id breaks ties
LIMIT 3 OFFSET 3;
```

Result

name	salary
Meera	80000
Neha	75000
Arjun	65000

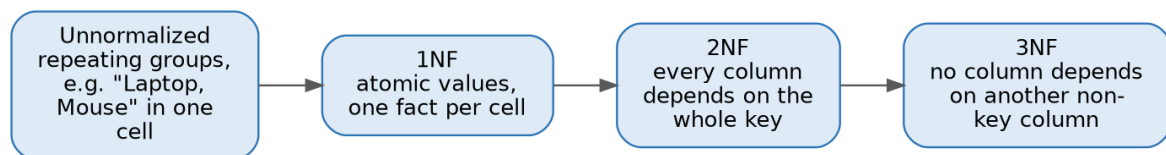
Interview tip: Large OFFSET values get slow because the database still reads and throws away the skipped rows. For deep paging, mention keyset pagination: `WHERE (salary, emp_id) < (last_salary, last_id) ORDER BY salary DESC, emp_id LIMIT 3.`

Q12. What is normalization, and when would you denormalize?

ANSWER

Normalization organizes tables so that each fact is stored once. It removes redundancy and prevents update, insert and delete anomalies (for example, changing a customer's city in one row but not another).

FLOW



Form	Rule	Example fix
1NF	Each cell holds one value; no repeating groups	Split "Laptop, Mouse" into two rows
2NF	Every non-key column depends on the whole key (matters for composite keys)	Move product_name out of order_items into products
3NF	No non-key column depends on another non-key column	Move city out of customers into a postal_codes table, if city is determined by postal code

Denormalization deliberately adds redundancy to make reads faster: pre-computed totals, copied columns, or wide reporting tables. It suits read-heavy workloads, analytics and data warehouses. The cost is more storage and the risk that copies drift out of sync, so you need a plan to keep them consistent.

2. Joins and Subqueries

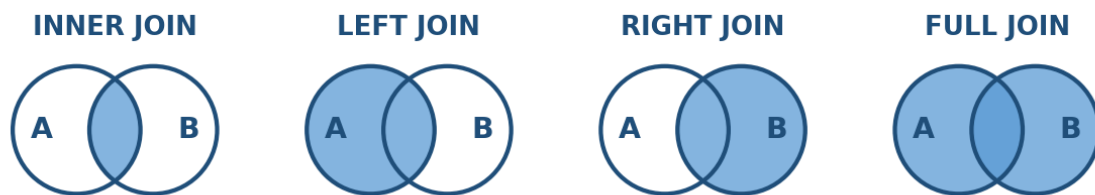
Joins are tested in nearly every SQL round. Know what each join returns for unmatched rows.

Q13. Explain INNER, LEFT, RIGHT and FULL joins.

ANSWER

A join combines rows from two tables based on a condition. The join type decides what happens to rows that have no match on the other side.

FLOW



A = employees (left table) B = departments (right table) shaded = rows returned

Join	Rows returned	Unmatched rows
INNER	Only rows that match in both tables	Dropped
LEFT (OUTER)	All rows from the left table, plus matches	Left rows kept, right columns are NULL
RIGHT (OUTER)	All rows from the right table, plus matches	Right rows kept, left columns are NULL
FULL (OUTER)	All rows from both tables	Kept from both sides, with NULLs

EXAMPLE

Our sample data has one employee with no department (Zoya) and one department with no employees (Legal), which makes the differences visible.

Sample data: departments

dept_id	dept_name
1	Engineering
2	Sales
3	HR
4	Legal

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id; -- or LEFT / RIGHT / FULL
```

Row counts (tested on the sample data)

Join type	Rows returned	Why
INNER	10	10 employees have a matching department
LEFT	11	10 matches + Zoya with dept_name NULL
RIGHT	11	10 matches + Legal with name NULL
FULL	12	10 matches + Zoya + Legal

```
SELECT e.name, d.dept_name
FROM employees e
FULL JOIN departments d ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL OR d.dept_id IS NULL;
```

FULL JOIN, unmatched rows only

name	dept_name
Zoya	NULL
NULL	Legal

Interview tip: MySQL has no FULL JOIN. Emulate it with LEFT JOIN ... UNION ... RIGHT JOIN. Also remember that a filter on the right table in the WHERE clause of a LEFT JOIN turns it into an inner join; put such conditions in the ON clause instead.

Q14. What is a CROSS join, and when is it useful?

ANSWER

A CROSS join returns the Cartesian product: every row of the first table paired with every row of the second, with no join condition. Two tables of 2 rows each give 4 rows; 1,000 by 1,000 rows give 1,000,000, so use it deliberately.

- Generate every combination, such as sizes x colors for a product catalogue.
- Build a calendar grid: every store x every date, then LEFT JOIN actual sales to show zeros for days with no sales.

EXAMPLE

```
WITH sizes(size) AS (VALUES ('S'), ('M')),
     colors(color) AS (VALUES ('Red'), ('Blue'))
SELECT size, color
FROM sizes CROSS JOIN colors;
```

2 sizes x 2 colors = 4 rows

size	color
S	Red
S	Blue
M	Red
M	Blue

An old-style join with a missing condition (**FROM a, b** and no WHERE) silently becomes a cross join, which is a classic cause of accidentally huge result sets.

Q15. What is a self join? Find employees who earn more than their manager.

ANSWER

A self join joins a table to itself, using two different aliases so the database can tell the two copies apart. It is the standard way to read hierarchies stored in one table, where each row holds the id of its parent (here, `manager_id`).

EXAMPLE

```
SELECT e.name AS employee, e.salary,
       m.name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Result

employee	salary	manager	manager_salary
Karan	85000	Ravi	80000

Karan (85,000) reports to Ravi (80,000). Employees with no manager (`manager_id` is NULL) drop out of the inner join. Use LEFT JOIN if you need to keep them.

Q16. How do you find rows in one table that have no match in another?

ANSWER

This is called an **anti-join**. There are three ways to write it. `NOT EXISTS` and `LEFT JOIN ... IS NULL` are safe. `NOT IN` is dangerous when the subquery can return NULL (see Q8).

EXAMPLE

Which departments have no employees?

```
-- Option 1: NOT EXISTS (recommended)
SELECT d.dept_name
FROM departments d
WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

Option 1 result

dept_name
Legal

```
-- Option 2: LEFT JOIN and keep the rows with no match
SELECT d.dept_name
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

Option 2 result (same)

dept_name
Legal

Interview tip: Be ready to explain why NOT IN fails here: `employees.dept_id` contains a NULL, so NOT IN returns no rows at all.

Q17. What is the difference between a JOIN and a subquery?

ANSWER

Both can express the same logic. A JOIN combines tables side by side, so you can select columns from both. A subquery is a query nested inside another one, and it normally only supplies values for filtering or calculation.

- Use a **JOIN** when you need columns from both tables in the output.
- Use a **subquery** (IN, EXISTS, scalar) when you only need to filter, or when it reads more clearly.
- Modern optimizers often turn one into the other, so performance is frequently identical. Always check the execution plan rather than assuming.
- A JOIN can multiply rows when the other side has several matches; EXISTS never does.

EXAMPLE

Employees who work in Sales, written both ways. Both return the same four people:

```
-- JOIN
SELECT e.name
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
WHERE d.dept_name = 'Sales';

-- Subquery
SELECT name
FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE dept_name = 'Sales');
```

Result

name
Neha
Arjun
Divya
Isha

Q18. What is the difference between a correlated and a nested subquery?

ANSWER

A **nested (non-correlated) subquery** runs on its own and its result does not depend on the outer query. Conceptually it runs once. A **correlated subquery** refers to a column of the outer query, so conceptually it is evaluated once for every outer row. That makes it flexible but potentially slow (see Q40 for how to rewrite it).

EXAMPLE

Nested: employees who earn more than the company average (one average for everyone).

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC, name;
```

Company average is about 68,182

name	salary
Asha	90000
Karan	85000
Meera	80000
Ravi	80000
Neha	75000

Correlated: employees who earn more than the average of their own department. The inner query uses **e.dept_id** from the outer row.

```
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary)
                  FROM employees
                  WHERE dept_id = e.dept_id)  -- refers to the outer row
ORDER BY e.name;
```

Result

name	salary
Asha	90000
Karan	85000
Neha	75000
Vikram	55000

Zoya has no department, so her inner query finds no peers, returns NULL, and the comparison is not true. That is why she is absent.

3. Hands-on Query Problems

These live-coding problems decide most interviews. Most of them are one of a handful of patterns (see the cheat sheet at the start).

Q19. Find the second highest salary.

ANSWER

This is the most famous SQL interview question. The key detail is **ties**: if two people share the top salary, the "second highest" should still be the next distinct value. Show two or three methods and say which one you prefer and why.

Method 1: DENSE_RANK (best general answer, easy to extend to Nth). DENSE_RANK gives tied rows the same rank without gaps.

```
WITH ranked AS (  
  SELECT salary,  
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk  
  FROM employees  
)  
SELECT DISTINCT salary AS second_highest_salary  
FROM ranked  
WHERE rnk = 2;
```

Result

second_highest_salary

85000

Method 2: MAX with a subquery (works everywhere, only for the second):

```
SELECT MAX(salary) AS second_highest_salary  
FROM employees  
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Result

second_highest_salary

85000

Method 3: LIMIT / OFFSET on distinct values. Wrapping it in a scalar subquery returns NULL, instead of no row, when there is no second value:

```
SELECT (SELECT DISTINCT salary  
        FROM employees  
        ORDER BY salary DESC  
        LIMIT 1 OFFSET 1) AS second_highest_salary;
```

Result

second_highest_salary

85000

Interview tip: Why not ORDER BY ... LIMIT 1 OFFSET 1 without DISTINCT? With ties at the top it would return the top salary again. Also say what you return when no second value exists (NULL).

Q20. Find the Nth highest salary.

ANSWER

Generalize the DENSE_RANK method: replace the 2 with N. Because DENSE_RANK has no gaps, rank N is always the Nth distinct salary.

EXAMPLE

Third highest distinct salary (N = 3). The two employees at 80,000 tie, and both count as rank 3:

```
WITH ranked AS (  
    SELECT salary,  
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk  
    FROM employees  
)  
SELECT DISTINCT salary AS nth_highest_salary  
FROM ranked  
WHERE rnk = 3;          -- put N here
```

Result

nth_highest_salary
80000

In a stored function or application, N becomes a parameter. If N is larger than the number of distinct salaries, the query returns no rows.

Q21. Find the second highest salary in each department.

ANSWER

Add **PARTITION BY dept_id** so that the ranking restarts inside every department. Use DENSE_RANK so that ties do not hide the second distinct value.

EXAMPLE

```
WITH ranked AS (  
    SELECT dept_id, name, salary,  
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk  
    FROM employees  
)  
SELECT dept_id, name, salary  
FROM ranked  
WHERE rnk = 2  
ORDER BY dept_id, name;
```

Result

dept_id	name	salary
1	Karan	85000
2	Arjun	65000
2	Divya	65000
3	Sana	50000

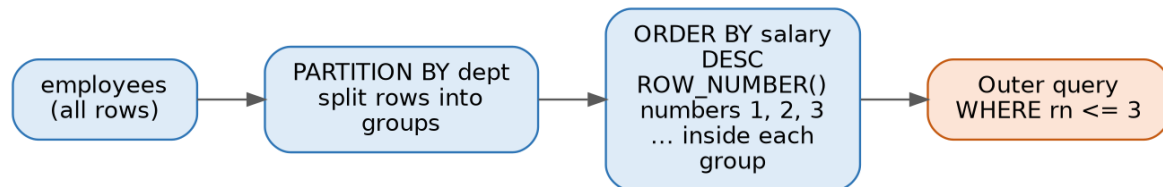
In Sales, Arjun and Divya tie at 65,000, so both are returned. Zoya has no department; she forms a group of one with rank 1, so she does not appear.

Q22. Find the top 3 earners in each department.

ANSWER

This is the **top-N per group** pattern. Rank the rows inside each department, then filter in an outer query, because window functions cannot be used directly in WHERE (see Q9).

FLOW



EXAMPLE

```
WITH ranked AS (  
  SELECT dept_id, name, salary,  
         ROW_NUMBER() OVER (PARTITION BY dept_id  
                             ORDER BY salary DESC, emp_id) AS rn  
  FROM employees  
  WHERE dept_id IS NOT NULL  
)  
SELECT dept_id, name, salary, rn  
FROM ranked  
WHERE rn <= 3  
ORDER BY dept_id, rn;
```

Result

dept_id	name	salary	rn
1	Asha	90000	1
1	Karan	85000	2
1	Ravi	80000	3
2	Neha	75000	1
2	Arjun	65000	2
2	Divya	65000	3
3	Vikram	55000	1
3	Sana	50000	2

- **ROW_NUMBER** returns exactly 3 rows per group, and breaks ties arbitrarily (hence the **emp_id** tie-breaker for repeatable results).
- **RANK** or **DENSE_RANK** would return everyone who ties into the top 3, which can be more than 3 rows.
- Ask the interviewer which behaviour they want. That question alone scores points.

Q23. Find duplicate rows (for example, duplicate emails).

ANSWER

Group by the column (or columns) that should be unique and keep the groups that appear more than once.

EXAMPLE

Sample data: customers

id	name	email
1	Aman	a@x.com
2	Bela	b@x.com
3	Chirag	a@x.com
4	Diya	d@x.com
5	Esha	b@x.com

```
SELECT email, COUNT(*) AS times_used
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

Result

email	times_used
a@x.com	2
b@x.com	2

To see the full duplicate rows instead of just the counts:

```
SELECT *
FROM customers
WHERE email IN (SELECT email FROM customers
                GROUP BY email HAVING COUNT(*) > 1)
ORDER BY email, id;
```

Result

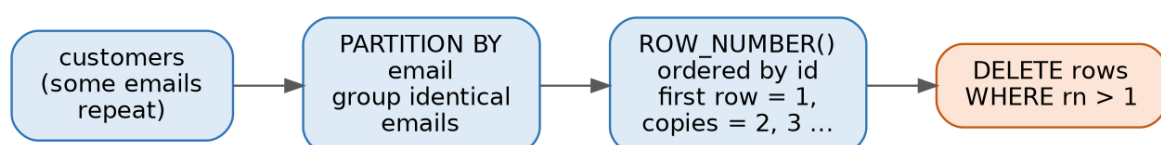
id	name	email
1	Aman	a@x.com
3	Chirag	a@x.com
2	Bela	b@x.com
5	Esha	b@x.com

Q24. Delete duplicate rows while keeping one.

ANSWER

Number the rows inside each group of duplicates, keep row number 1 (for example the one with the lowest id), and delete the rest. Always run the SELECT version first to preview what will be deleted, and run the DELETE inside a transaction.

FLOW



EXAMPLE

```
DELETE FROM customers
WHERE id IN (
  SELECT id
  FROM (
    SELECT id,
           ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
    FROM customers
  ) t
  WHERE rn > 1
);
```

Rows to be deleted (the preview):

Preview: rows with rn > 1

id	email
3	a@x.com
5	b@x.com

Table after running the DELETE (ids 3 and 5 are gone):

After the DELETE

id	name	email
1	Aman	a@x.com
2	Bela	b@x.com
4	Diya	d@x.com

Interview tip: The derived table (**t**) matters: MySQL does not allow selecting from the table you are deleting from unless the subquery is wrapped like this. To prevent duplicates in future, add a UNIQUE constraint on email.

Q25. Calculate a running total of sales by date.

ANSWER

Use SUM() as a window function with ORDER BY. The frame **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** means "from the first row up to this one". Writing the frame explicitly is better than relying on the default, which uses RANGE and treats rows with the same date as one group (see Q38).

EXAMPLE

Sample data: sales

sale_date	amount
2026-01-01	100
2026-01-02	150
2026-01-03	120
2026-01-04	200
2026-01-05	180
2026-01-06	220
2026-01-07	160
2026-01-08	240

```
SELECT sale_date, amount,
       SUM(amount) OVER (
         ORDER BY sale_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM sales
ORDER BY sale_date;
```

Result

sale_date	amount	running_total
2026-01-01	100	100
2026-01-02	150	250
2026-01-03	120	370
2026-01-04	200	570
2026-01-05	180	750
2026-01-06	220	970
2026-01-07	160	1130
2026-01-08	240	1370

Add **PARTITION BY store_id** if you need a separate running total per store.

Q26. Calculate a 7-day moving average.

ANSWER

A moving average uses a sliding frame: the current row plus the 6 rows before it. This assumes one row per day with no missing dates. If dates can be missing, first join to a calendar table, or use a RANGE frame with an interval where your database supports it.

EXAMPLE

```
SELECT sale_date, amount,
       ROUND(AVG(amount) OVER (
         ORDER BY sale_date
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 2) AS moving_avg_7,
       COUNT(*) OVER (
         ORDER BY sale_date
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS days_in_window
FROM sales
ORDER BY sale_date;
```

Result

sale_date	amount	moving_avg_7	days_in_window
2026-01-01	100	100	1
2026-01-02	150	125	2
2026-01-03	120	123.33	3
2026-01-04	200	142.5	4
2026-01-05	180	150	5
2026-01-06	220	161.67	6
2026-01-07	160	161.43	7
2026-01-08	240	181.43	7

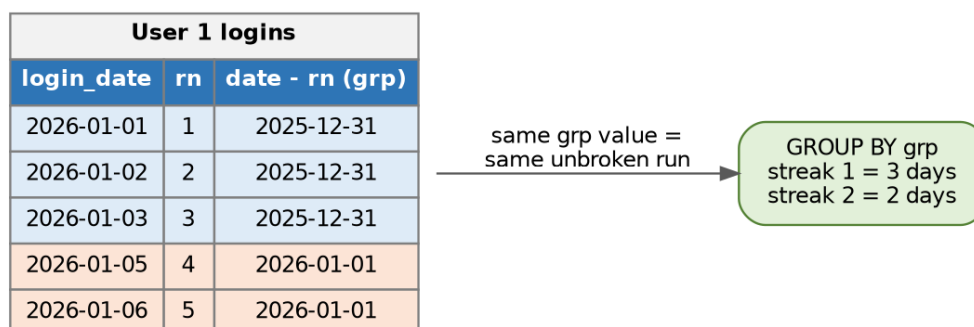
The first six rows use a partial window (see `days_in_window`). If you only want full 7-day averages, filter out the rows where `days_in_window < 7`.

Q27. Find users who logged in on 3 or more consecutive days.

ANSWER

This is the **gaps and islands** pattern, and the trick is "date minus row number". Number each user's distinct login dates 1, 2, 3 ... in order. For consecutive dates, the date and the row number rise together, so their difference stays constant. A gap changes the difference. Rows with the same difference form one unbroken streak.

FLOW



EXAMPLE

Sample data: logins

user_id	login_date
1	2026-01-01
1	2026-01-02
1	2026-01-03
1	2026-01-05
1	2026-01-06
2	2026-01-02
2	2026-01-03
2	2026-01-04
2	2026-01-05
2	2026-01-06
2	2026-01-08

```
-- PostgreSQL (login_date is a DATE)
WITH numbered AS (
  SELECT user_id, login_date,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS rn
  FROM (SELECT DISTINCT user_id, login_date FROM logins) d
),
ranked AS (
  SELECT user_id, login_date,
         login_date - CAST(rn AS INT) AS grp      -- constant inside one streak
  FROM numbered
)
SELECT user_id,
       MIN(login_date) AS streak_start,
       MAX(login_date) AS streak_end,
       COUNT(*)        AS days
FROM ranked
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

Result

user_id	streak_start	streak_end	days
1	2026-01-01	2026-01-03	3
2	2026-01-02	2026-01-06	5

- MySQL: `DATE_SUB(login_date, INTERVAL rn DAY)`. SQL Server: `DATEADD(DAY, -rn, login_date)`.
- The **DISTINCT** step matters: two logins on the same day would otherwise break the arithmetic.
- The query was tested on the sample data using SQLite date functions, so the logic is verified; only the date arithmetic differs between databases.

Q28. Find the longest login streak per user.

ANSWER

Reuse the streak query from Q27 without the HAVING filter, then take the maximum streak length per user in an outer query.

EXAMPLE

```
WITH numbered AS (  
    SELECT user_id, login_date,  
           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS rn  
    FROM (SELECT DISTINCT user_id, login_date FROM logins) d  
)  
,  
ranked AS (  
    SELECT user_id, login_date, login_date - CAST(rn AS INT) AS grp  
    FROM numbered  
)  
,  
streaks AS (  
    SELECT user_id, grp, COUNT(*) AS days  
    FROM ranked  
    GROUP BY user_id, grp  
)  
SELECT user_id, MAX(days) AS longest_streak  
FROM streaks  
GROUP BY user_id;
```

Result

user_id	longest_streak
1	3
2	5

User 1 has streaks of 3 and 2 days; user 2 has streaks of 5 and 1 day. The "current streak" is the streak whose last date is today (or yesterday).

Q29. Find missing IDs or missing dates in a sequence.

ANSWER

For **missing IDs**, compare each id with the next one using LEAD(). If the next id is more than 1 higher, everything in between is missing.

EXAMPLE

Sample data: seq_ids

id
1
2
3
6
7
10

```

SELECT id + 1      AS gap_start,
       next_id - 1 AS gap_end
FROM (
  SELECT id, LEAD(id) OVER (ORDER BY id) AS next_id
  FROM seq_ids
) t
WHERE next_id - id > 1;

```

Result

gap_start	gap_end
4	5
8	9

The missing ids are 4 to 5 and 8 to 9. For **missing dates**, generate a full calendar (a calendar table, **generate_series** in PostgreSQL, or a recursive CTE) and LEFT JOIN your data to it. Dates where the joined column is NULL are the missing ones.

Q30. Calculate month-over-month change using LAG().

ANSWER

LAG(column) reads a value from the previous row in the window order. Subtract it from the current value for the change, and divide by it for the percentage. The first month has no previous value, so the result is NULL.

EXAMPLE

Sample data: *monthly_sales*

month	revenue
2026-01	1000
2026-02	1200
2026-03	900
2026-04	1500

```

SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month)      AS prev_revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS change,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
            / LAG(revenue) OVER (ORDER BY month), 1) AS pct_change
FROM monthly_sales
ORDER BY month;

```

Result

month	revenue	prev_revenue	change	pct_change
2026-01	1000	NULL	NULL	NULL
2026-02	1200	1000	200	20
2026-03	900	1200	-300	-25
2026-04	1500	900	600	66.7

LEAD() is the mirror image and looks at the next row. **LAG(revenue, 12)** compares with the same month last year. Guard against division by zero with **NULLIF(LAG(revenue) OVER (...), 0)**.

Q31. Calculate the median salary.

ANSWER

Most databases have no simple MEDIAN function. The cleanest answer is the percentile function where it exists. The portable alternative numbers the rows and picks the middle one (or averages the two middle ones when the count is even).

EXAMPLE

PostgreSQL (also Oracle and SQL Server with small syntax differences):

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary
FROM employees;
```

Portable version using window functions:

```
SELECT AVG(salary) AS median_salary
FROM (
  SELECT salary,
         ROW_NUMBER() OVER (ORDER BY salary) AS rn,
         COUNT(*) OVER () AS cnt
  FROM employees
) t
WHERE rn IN ((cnt + 1) / 2, (cnt + 2) / 2);
```

Result

median_salary
65000

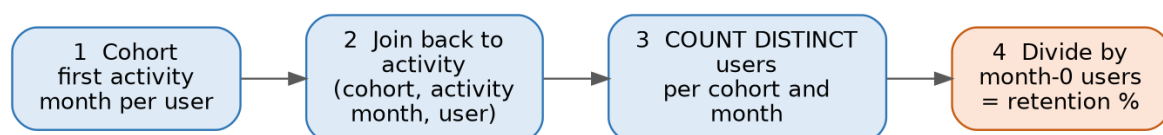
There are 11 employees, so the middle row is number 6. With an even count, the two middle rows are averaged. The integer division in $(cnt + 1) / 2$ is what picks one row or two.

Q32. Calculate customer retention by cohort.

ANSWER

Retention asks: of the users who first appeared in a given month (their **cohort**), how many are still active in later months? Find each user's first month, join it back to their activity, count distinct users per cohort and month, then divide by the cohort's size in its first month.

FLOW



EXAMPLE

Sample data: activity

user_id	activity_month
1	2026-01
1	2026-02
1	2026-03
2	2026-01
2	2026-02
3	2026-01
4	2026-01
4	2026-03
5	2026-02
5	2026-03
6	2026-02

```
WITH first_month AS (  
    SELECT user_id, MIN(activity_month) AS cohort_month  
    FROM activity  
    GROUP BY user_id  
),  
counts AS (  
    SELECT f.cohort_month, a.activity_month,  
           COUNT(DISTINCT a.user_id) AS active_users  
    FROM first_month f  
    JOIN activity a ON a.user_id = f.user_id  
    GROUP BY f.cohort_month, a.activity_month  
),  
sized AS (  
    SELECT *,  
           FIRST_VALUE(active_users) OVER (  
               PARTITION BY cohort_month  
               ORDER BY activity_month) AS cohort_size  
    FROM counts  
)  
SELECT cohort_month, activity_month, active_users,  
       ROUND(100.0 * active_users / cohort_size, 1) AS retention_pct  
FROM sized  
ORDER BY cohort_month, activity_month;
```

Result

cohort_month	activity_month	active_users	retention_pct
2026-01	2026-01	4	100
2026-01	2026-02	2	50
2026-01	2026-03	2	50
2026-02	2026-02	2	100
2026-02	2026-03	1	50

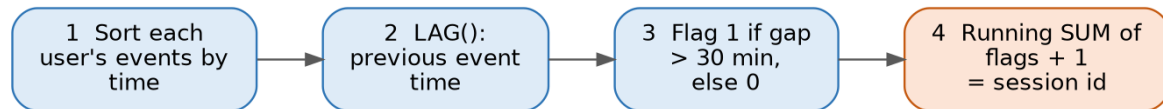
The January cohort has 4 users. Two of them are active in February and two in March, so retention is 50% in both months. Pivot the result into a grid (cohort by month) for a classic retention table.

Q33. Group events into sessions (sessionization).

ANSWER

A session is a run of events with no long pause between them. Use LAG() to measure the gap to the previous event, flag a new session when the gap exceeds a threshold (here 30 minutes), and take a running SUM of the flags to turn them into a session number.

FLOW



EXAMPLE

Sample data: events (event_minute = minutes since midnight)

user_id	event_minute
1	0
1	5
1	12
1	60
1	70
1	200
2	10
2	15

```
WITH flagged AS (  
    SELECT user_id, event_minute,  
           CASE WHEN event_minute - LAG(event_minute) OVER (  
               PARTITION BY user_id ORDER BY event_minute) > 30  
               THEN 1 ELSE 0 END AS new_session  
    FROM events  
)  
SELECT user_id, event_minute,  
       SUM(new_session) OVER (  
           PARTITION BY user_id ORDER BY event_minute) + 1 AS session_id  
FROM flagged  
ORDER BY user_id, event_minute;
```

Result

user_id	event_minute	session_id
1	0	1
1	5	1
1	12	1
1	60	2
1	70	2
1	200	3
2	10	1
2	15	1

User 1 has three sessions: minutes 0 to 12, minutes 60 to 70, and minute 200. With real timestamps, replace the subtraction with `event_time - LAG(event_time) OVER (...) > INTERVAL '30 minutes'` in PostgreSQL.

Q34. Pivot rows into columns, and unpivot columns into rows.

ANSWER

Pivot turns the values of one column (here the quarter) into separate columns. The portable way is conditional aggregation: `SUM(CASE WHEN ... END)` with `GROUP BY`. SQL Server and Oracle also have a `PIVOT` keyword, and PostgreSQL has the `crosstab` extension. **Unpivot** does the reverse; the portable way is `UNION ALL`, one `SELECT` per column.

EXAMPLE

Sample data: `product_sales`

product	quarter	revenue
Laptop	Q1	100
Laptop	Q2	120
Phone	Q1	80
Phone	Q2	90

Pivot (rows to columns):

```
SELECT product,
       SUM(CASE WHEN quarter = 'Q1' THEN revenue END) AS q1,
       SUM(CASE WHEN quarter = 'Q2' THEN revenue END) AS q2
FROM product_sales
GROUP BY product
ORDER BY product;
```

Result

product	q1	q2
Laptop	100	120
Phone	80	90

Unpivot (columns back to rows), starting from the wide table above:

```
SELECT product, 'Q1' AS quarter, q1 AS revenue FROM quarterly_wide
UNION ALL
SELECT product, 'Q2',          q2          FROM quarterly_wide
ORDER BY product, quarter;
```

Result

product	quarter	revenue
Laptop	Q1	100
Laptop	Q2	120
Phone	Q1	80
Phone	Q2	90

4. Window Functions and CTEs

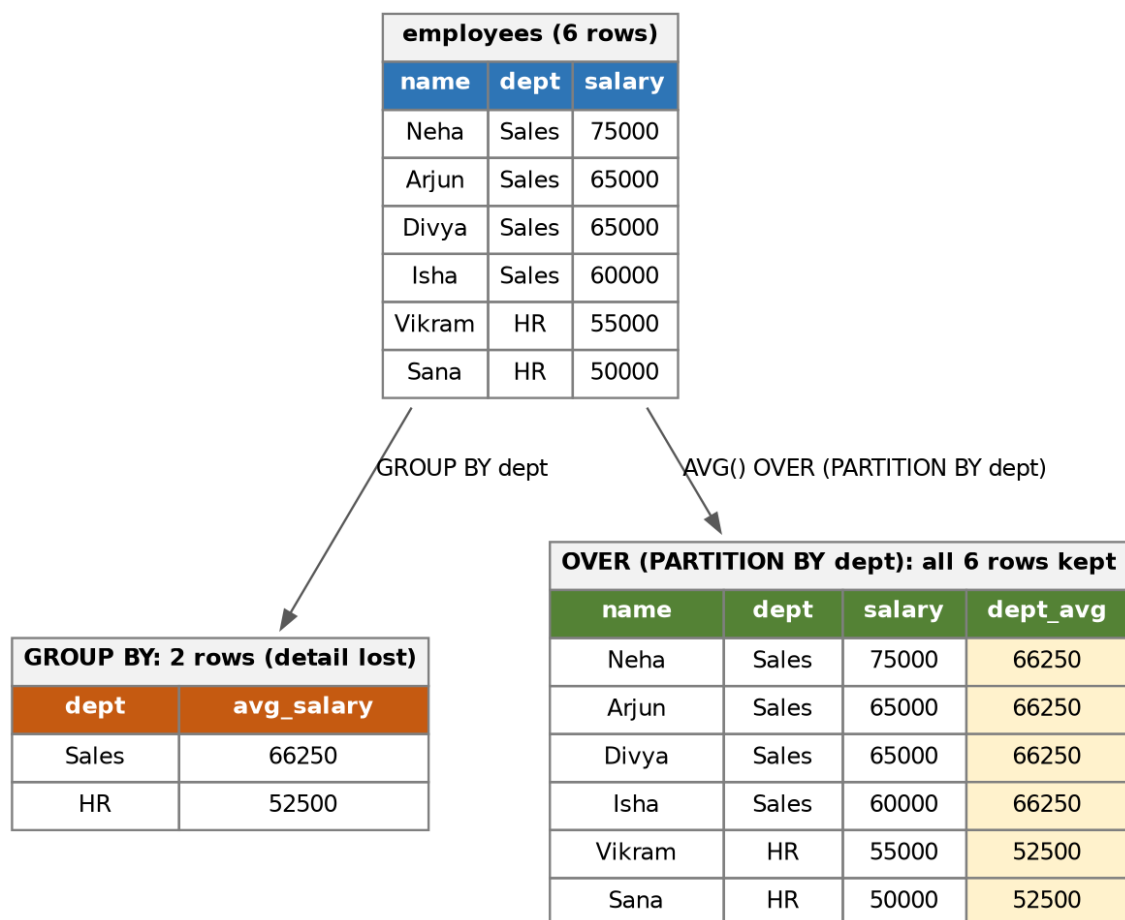
The most commonly tested advanced topic. Window functions let you keep row detail while adding calculations across rows.

Q35. What is a window function, and how does it differ from GROUP BY?

ANSWER

A window function calculates a value across a set of related rows (the "window") **without collapsing them**. GROUP BY returns one row per group and the row-level detail is lost. A window function keeps every row and adds a new column. The window is defined with **OVER (PARTITION BY ... ORDER BY ... frame)**.

FLOW



EXAMPLE

Each employee next to the average salary of their own department (Sales and HR shown):

```
SELECT name, dept_id, salary,
       ROUND(AVG(salary) OVER (PARTITION BY dept_id)) AS dept_avg
FROM employees
WHERE dept_id IN (2, 3)
ORDER BY dept_id, emp_id;
```

Result

name	dept_id	salary	dept_avg
Neha	2	75000	66250
Arjun	2	65000	66250
Divya	2	65000	66250
Isha	2	60000	66250
Vikram	3	55000	52500
Sana	3	50000	52500

With GROUP BY you would get only two rows (one per department) and could not show Neha's salary next to the average without a join. Families of window functions: ranking (ROW_NUMBER, RANK, DENSE_RANK, NTILE), value (LAG, LEAD, FIRST_VALUE, LAST_VALUE) and aggregate (SUM, AVG, COUNT, MIN, MAX with OVER).

Q36. What is the difference between RANK, DENSE_RANK and ROW_NUMBER?

ANSWER

All three number the rows in a window. They differ only in how they treat ties.

- **ROW_NUMBER**: always unique (1, 2, 3, 4 ...). Tied rows get different numbers, chosen arbitrarily unless you add a tie-breaker.
- **RANK**: tied rows share a rank, and the next rank is skipped (1, 2, 2, 4).
- **DENSE_RANK**: tied rows share a rank, and no rank is skipped (1, 2, 2, 3).

EXAMPLE

Top five salaries. Ravi and Meera tie at 80,000:

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC, emp_id) AS row_num,
       RANK() OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees
ORDER BY salary DESC, emp_id
LIMIT 5;
```

Result

name	salary	row_num	rnk	dense_rnk
Asha	90000	1	1	1
Karan	85000	2	2	2
Ravi	80000	3	3	3
Meera	80000	4	3	3
Neha	75000	5	5	4

Use case	Best choice
Nth highest value	DENSE_RANK
Top N including ties	RANK (or DENSE_RANK)
Exactly N rows, remove duplicates, pagination	ROW_NUMBER

Q37. What does PARTITION BY do?

ANSWER

PARTITION BY splits the rows into independent groups for the window function, much like GROUP BY, except that rows are not collapsed. The function restarts for every partition: numbering restarts at 1, running totals restart at 0, and LAG() does not look across partition borders. Without PARTITION BY, the whole result set is one window.

EXAMPLE

Rank inside each department (Engineering and HR shown):

```
SELECT name, dept_id, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
FROM employees
WHERE dept_id IN (1, 3)
ORDER BY dept_id, dept_rank, name;
```

Result

name	dept_id	salary	dept_rank
Asha	1	90000	1
Karan	1	85000	2
Meera	1	80000	3
Ravi	1	80000	3
Vikram	3	55000	1
Sana	3	50000	2

Note the restart: Vikram is rank 1 in HR even though his salary is lower than every Engineering salary.

Q38. How do window frame clauses (ROWS BETWEEN) work?

ANSWER

The **frame** decides which rows, inside the partition, the function sees for the current row. It is written as **ROWS** (or **RANGE**) **BETWEEN** *start* **AND** *end*, where the boundaries are **UNBOUNDED PRECEDING**, *n* **PRECEDING**, **CURRENT ROW**, *n* **FOLLOWING** and **UNBOUNDED FOLLOWING**.

Frame	Meaning	Typical use
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	From the first row to this row	Running total
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW	This row and the 6 before it	7-row moving average
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING	Previous, current and next row	Smoothing
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	The whole partition	LAST_VALUE, share of total

ROWS vs RANGE. ROWS counts physical rows. RANGE groups together all rows that have the same ORDER BY value (peers). When you write ORDER BY and no frame, most databases default to **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**, which surprises people when there are ties:

EXAMPLE

```
SELECT name, salary,
       SUM(salary) OVER (ORDER BY salary) AS default_range_sum,
       SUM(salary) OVER (ORDER BY salary
                        ROWS BETWEEN UNBOUNDED PRECEDING
                        AND CURRENT ROW) AS rows_sum
FROM employees
WHERE dept_id = 2
ORDER BY salary, emp_id;
```

Result

name	salary	default_range_sum	rows_sum
Isha	60000	60000	60000
Arjun	65000	190000	125000
Divya	65000	190000	190000
Neha	75000	265000	265000

Arjun and Divya both earn 65,000. With the default RANGE frame, they are peers, so both rows show 190,000. With ROWS, the sum grows one row at a time (125,000, then 190,000). That is why an explicit ROWS frame is the safer choice for running totals.

Q39. What is a CTE, and when would you use one over a subquery?

ANSWER

A Common Table Expression (CTE) is a named temporary result defined with **WITH name AS (...)** and visible only to the statement that follows. Use a CTE to break a long query into readable steps, to reuse the same result more than once, and to write recursive queries. A subquery is fine for a short, one-off filter.

EXAMPLE

Departments whose average salary is above the company average, built in named steps:

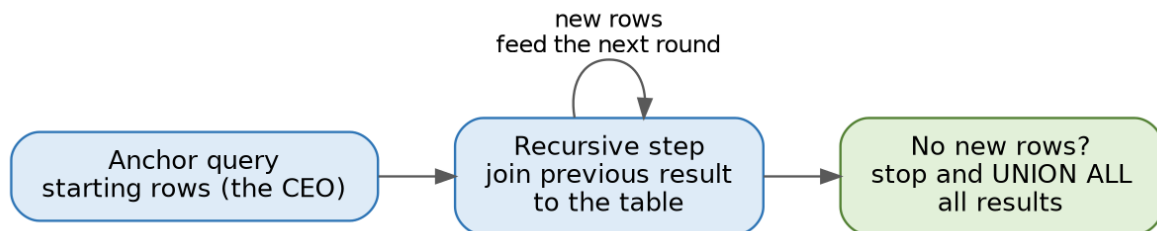
```
WITH dept_avg AS (
  SELECT dept_id, AVG(salary) AS avg_sal
  FROM employees
  GROUP BY dept_id
),
company AS (
  SELECT AVG(salary) AS avg_all FROM employees
)
SELECT d.dept_name, ROUND(da.avg_sal) AS avg_salary
FROM dept_avg da
JOIN departments d ON d.dept_id = da.dept_id
CROSS JOIN company c
WHERE da.avg_sal > c.avg_all
ORDER BY avg_salary DESC;
```

Result

dept_name	avg_salary
Engineering	83750

Recursive CTEs walk hierarchies such as org charts, folders or category trees. They have an anchor query plus a recursive part that joins the previous result back to the table.

FLOW



```

WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 1 AS level
  FROM employees
  WHERE emp_id = 1                -- anchor: start at Asha
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, o.level + 1
  FROM employees e
  JOIN org o ON e.manager_id = o.emp_id -- recursive step
)
SELECT name, level FROM org ORDER BY level, name;
  
```

Everyone in Asha's reporting chain

name	level
Asha	1
Meera	2
Ravi	2
Karan	3

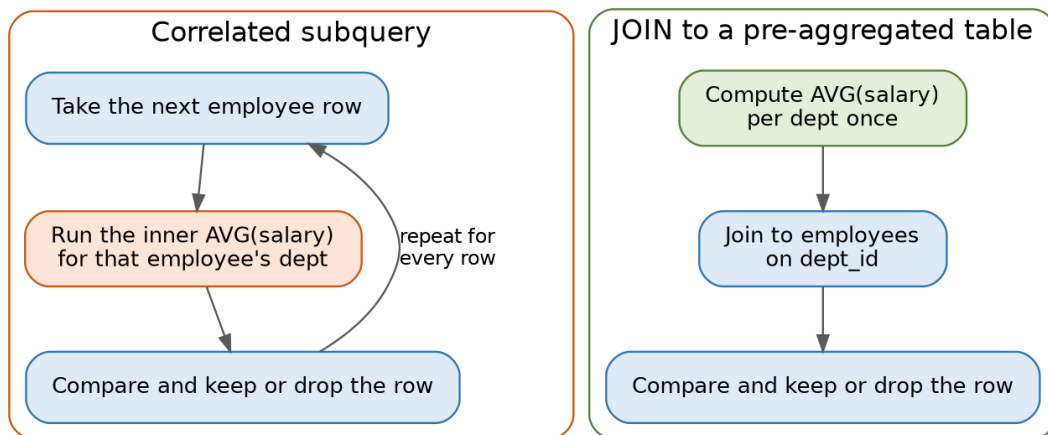
SQL Server and Oracle omit the word RECURSIVE. Note that, in some databases (older MySQL and PostgreSQL versions before 12), a CTE may be computed once and not inlined, which can matter for performance.

Q40. How do you rewrite a correlated subquery as a JOIN?

ANSWER

A correlated subquery is conceptually run once per outer row. To avoid that, compute the inner result **once for all groups** in a derived table (or CTE), then join it back to the outer table on the correlation column.

FLOW



EXAMPLE

Employees who earn more than their department average. First the correlated version, then the JOIN rewrite. Both return the same rows:

```
-- Correlated subquery
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees
                  WHERE dept_id = e.dept_id);

-- Rewrite: aggregate once, then join
SELECT e.name, e.salary
FROM employees e
JOIN (SELECT dept_id, AVG(salary) AS avg_sal
      FROM employees
      GROUP BY dept_id) a ON a.dept_id = e.dept_id
WHERE e.salary > a.avg_sal;
```

Result of the JOIN version (identical to Q18)

name	salary
Asha	90000
Karan	85000
Neha	75000
Vikram	55000

A window function gives a third option: **AVG(salary) OVER (PARTITION BY dept_id)** in a CTE, then filter on it. Many optimizers rewrite simple correlated subqueries automatically, so always compare execution plans before claiming a speed-up.

5. Performance and Indexing

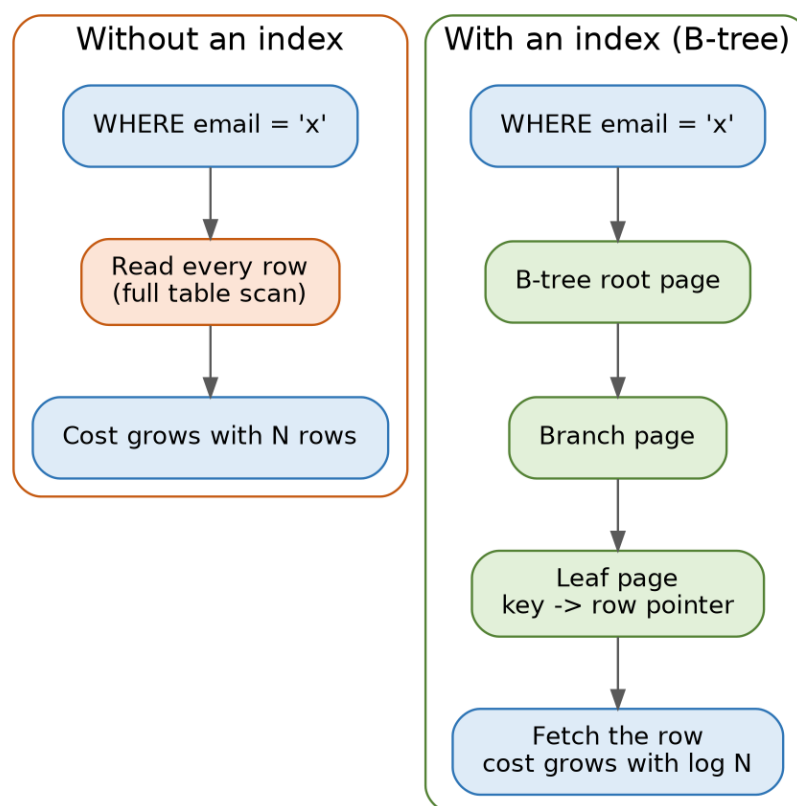
Expect these in backend, data engineering and DBA rounds. Always back claims with the execution plan.

Q41. What is an index, and how does it speed up queries?

ANSWER

An index is a separate sorted structure (usually a B-tree) that maps column values to the location of the rows. Without it, the database must read every row (a **full table scan**) to find matches. With it, the database walks a few levels of the tree straight to the matching entries, so the work grows with $\log N$ instead of N .

FLOW



EXAMPLE

```
CREATE INDEX idx_email ON big(email);
```

A real query plan from a 5,000-row test table, before and after creating the index (SQLite output; PostgreSQL shows "Seq Scan" and "Index Scan" instead):

Before the index

```
SCAN big
```

After the index

```
SEARCH big USING INDEX idx_email (email=?)
```

- Indexes on primary keys and UNIQUE constraints are created automatically in most databases.
- Index columns that appear often in WHERE, JOIN ... ON, ORDER BY and GROUP BY.

- A **composite index** (a, b) can serve queries on **a** alone or on **a** and **b** together, but not on **b** alone (the leftmost-prefix rule).

Q42. When can an index hurt performance?

ANSWER

Indexes make reads faster but are not free. Every INSERT, UPDATE and DELETE must also update each index on the table, and indexes take disk space and memory.

- **Write-heavy tables:** each extra index slows every write.
- **Low selectivity columns** (for example a boolean or a status with 3 values): the database often ignores the index because reading most of the table is cheaper than jumping around through it.
- **Small tables:** a full scan of a few pages is as fast as an index lookup.
- **Redundant or unused indexes:** (a) and (a, b) overlap, and indexes nobody queries waste space and write time. Review usage statistics and drop them.
- **Wide indexes** with many or very long columns are large and slow to maintain.

Interview tip: A strong answer ends with a rule of thumb: "Index for the queries you actually run, check the plan, and remove indexes that are not used."

Q43. Why should you avoid wrapping an indexed column in a function in WHERE?

ANSWER

The index stores the raw column values in sorted order. If you apply a function to the column, the database would have to compute the function for every row before it can compare, so it cannot use the index to jump to the answer. Such a condition is called **non-sargable** (not Search ARGument ABLE). The fix is to leave the column alone and move the work to the other side of the comparison, or to create an expression index.

Slow (non-sargable)	Fast (sargable)
WHERE YEAR(order_date) = 2026	WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01'
WHERE order_date + 7 > CURRENT_DATE	WHERE order_date > CURRENT_DATE - 7
WHERE SUBSTRING(email, 1, 5) = 'admin'	WHERE email LIKE 'admin%'
WHERE name LIKE '%son' (leading wildcard)	Use a full-text or trigram index

EXAMPLE

The column **email** is indexed, but wrapping it in **lower()** defeats the index:

WHERE lower(email) = ... (index not usable)

```
SCAN big
```

Creating an index on the expression itself fixes it (PostgreSQL and Oracle support function-based indexes, SQL Server uses computed columns, MySQL 8 supports functional indexes):

```
CREATE INDEX idx_lower_email ON big (lower(email));
```

After the expression index

```
SEARCH big USING INDEX idx_lower_email (<expr>=?)
```

Other common causes: implicit type conversion (comparing a text column to a number) and a leading wildcard in LIKE.

Q44. How do you read an execution plan (EXPLAIN)?

ANSWER

EXPLAIN shows how the database intends to run a query: which tables are read, in which order, with which access method and join algorithm, and what it expects each step to cost. **EXPLAIN ANALYZE** (PostgreSQL, MySQL 8) **actually runs** the query and adds real timings and row counts, so be careful with it on statements that change data. Read the plan from the innermost, most indented step outward, and compare estimated rows with actual rows.

Look for	What it means	Action
Seq Scan / Full Table Scan	Reads every row	Fine for small tables; for big ones, check for a missing index or a non-sargable filter
Index Scan / Index Seek	Uses an index to find rows	Usually good
Index Only Scan	Answers from the index without touching the table	Best case for reads
Nested Loop join	For each outer row, look up matches	Good for few rows with an index on the inner side
Hash Join	Builds a hash table from one input	Good for large, unsorted inputs
Merge Join	Merges two sorted inputs	Good when both are already sorted
Sort / big temp files	Sorting more data than fits in memory	Add an index matching the ORDER BY, or reduce rows earlier
Estimated rows far from actual	Statistics are stale	Run ANALYZE / update statistics

EXAMPLE

A real plan for a small join (SQLite output), showing one table scanned and the other searched by index:

EXPLAIN QUERY PLAN for a join

```
SCAN d
SEARCH e USING INDEX idx_e_dept (dept_id=?)
```

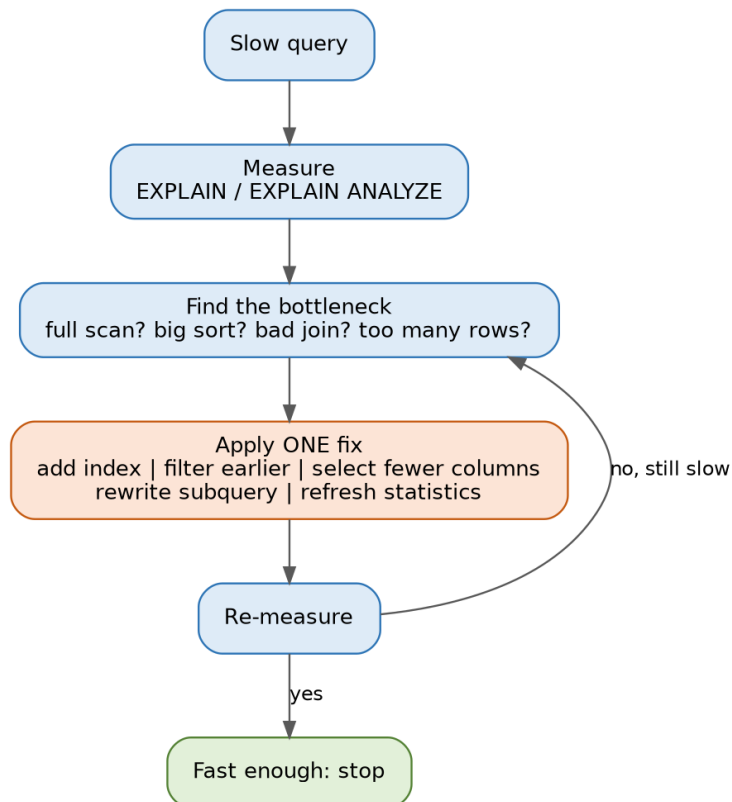
In PostgreSQL the same idea appears as lines such as **Nested Loop**, **Index Scan using ...** and **cost=0.15..8.17 rows=1 width=36**. The cost numbers are in arbitrary planner units, useful only to compare plans.

Q45. How would you optimize a slow query?

ANSWER

Do not guess. Measure, find the bottleneck, change one thing, and measure again. The loop below is the structure interviewers like to hear.

FLOW



- **Select only what you need.** Avoid `SELECT *`; fewer columns means less I/O and allows index-only scans.
- **Filter early.** Put conditions in WHERE, and keep them sargable (Q43).
- **Index the join and filter columns** that the plan shows being scanned.
- **Avoid per-row work.** Rewrite correlated subqueries as joins (Q40) and avoid functions on indexed columns.
- **Use UNION ALL instead of UNION** when duplicates are impossible or acceptable.
- **Fix pagination.** Replace deep OFFSET with keyset pagination.
- **Check statistics.** Stale statistics lead to bad plans; run ANALYZE.
- **Consider the data model.** Pre-aggregate with a summary table or materialized view for dashboards that repeat the same heavy query.

Interview tip: Always mention that you would test on realistic data volumes. A query that is instant on 1,000 rows can be very slow on 100 million.

6. Transactions and Design

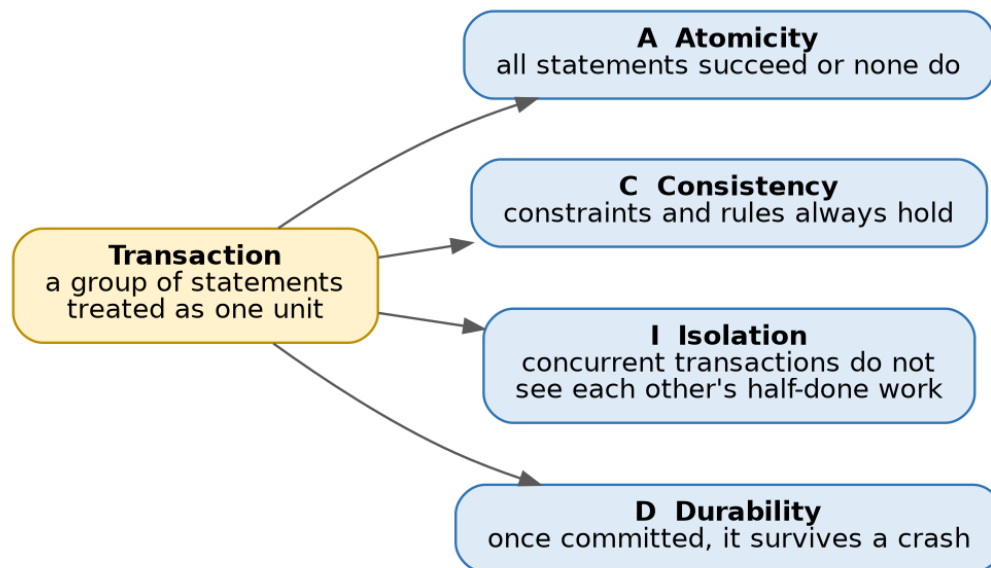
Often under-prepared, yet they appear consistently in backend and DBA rounds.

Q46. What are the ACID properties?

ANSWER

ACID describes the guarantees a database gives for a **transaction**, which is a group of statements treated as one unit of work.

FLOW



Property	Meaning	Bank transfer example
Atomicity	All steps succeed or none do	If crediting B fails, the debit from A is undone
Consistency	The database moves from one valid state to another; constraints always hold	Total money is unchanged; no negative balance if forbidden
Isolation	Concurrent transactions behave as if they ran one after another (to a degree set by the isolation level)	Another session never sees A debited but B not yet credited
Durability	Committed data survives crashes (write-ahead log)	After COMMIT, a power cut does not lose the transfer

EXAMPLE

Sample data: accounts

acc_id	balance
A	500
B	300

```
BEGIN;
UPDATE accounts SET balance = balance - 200 WHERE acc_id = 'A';
UPDATE accounts SET balance = balance + 200 WHERE acc_id = 'B';
COMMIT;      -- use ROLLBACK; instead to undo both updates
```

Balances after COMMIT

acc_id	balance
A	300
B	500

Balances after ROLLBACK (nothing changed)

acc_id	balance
A	500
B	300

Q47. What are transaction isolation levels, and what is a dirty or phantom read?

ANSWER

Isolation levels trade safety for concurrency. Higher levels prevent more anomalies but allow less parallel work. The anomalies are:

- **Dirty read:** you read data another transaction has changed but not yet committed (it may still be rolled back).
- **Non-repeatable read:** you read the same row twice in one transaction and get different values because someone committed a change in between.
- **Phantom read:** you run the same range query twice and the second run returns extra (or fewer) rows because someone inserted or deleted matching rows.

Isolation level (SQL standard)	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

This table is what the SQL standard allows. Real databases differ: PostgreSQL never allows dirty reads (its READ UNCOMMITTED behaves like READ COMMITTED), and its REPEATABLE READ also prevents phantoms. The default level is READ COMMITTED in PostgreSQL, SQL Server and Oracle, and REPEATABLE READ in MySQL InnoDB.

EXAMPLE

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ; -- syntax varies
BEGIN;
SELECT balance FROM accounts WHERE acc_id = 'A'; -- 500
-- meanwhile another session commits: balance = 400
SELECT balance FROM accounts WHERE acc_id = 'A'; -- still 500 here
COMMIT;
```

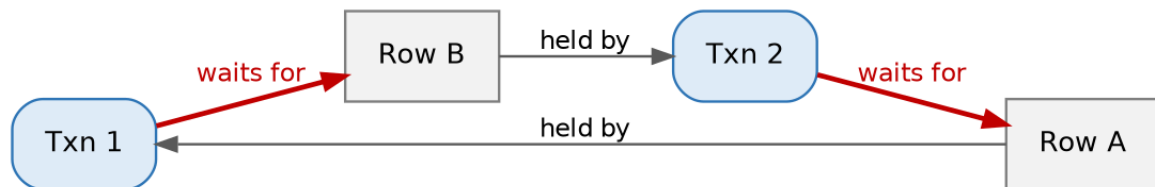
Interview tip: Say which level you would pick: READ COMMITTED for most applications, REPEATABLE READ or SERIALIZABLE for financial or reporting logic that needs a stable snapshot, and be ready to handle serialization failures with a retry.

Q48. What is a deadlock, and how do you prevent it?

ANSWER

A deadlock happens when two (or more) transactions each hold a lock the other one needs, so each waits forever for the other. The database detects the cycle and aborts one transaction (the "victim") with an error, which the application should catch and retry.

FLOW



Step	Transaction 1	Transaction 2
1	UPDATE row A (locks A)	
2		UPDATE row B (locks B)
3	UPDATE row B (waits for T2)	
4		UPDATE row A (waits for T1): deadlock

How to prevent or reduce deadlocks:

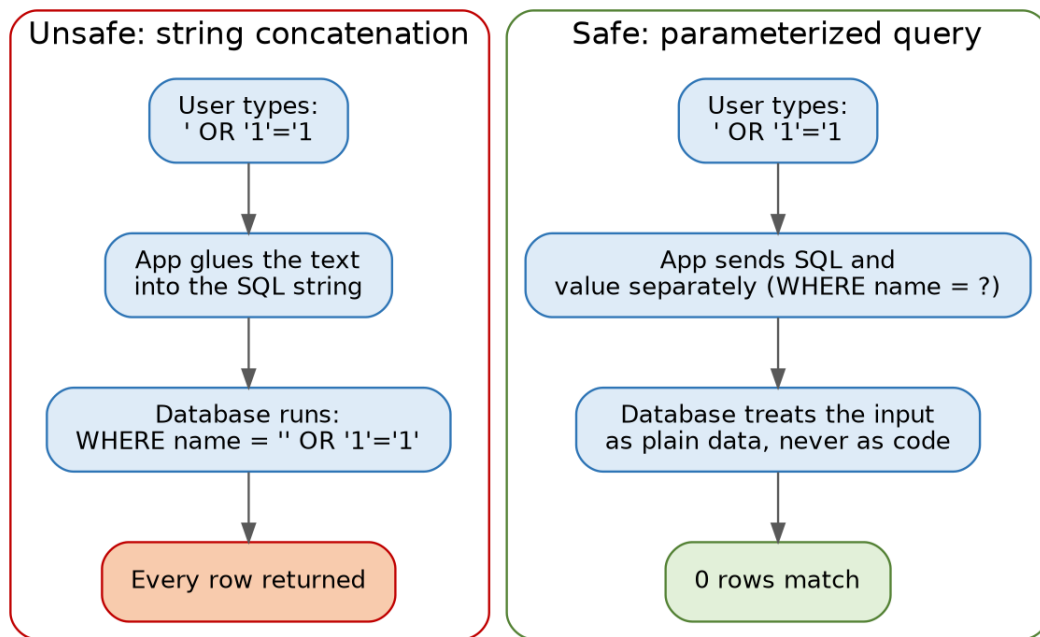
- Always touch tables and rows in the **same order** in every transaction.
- Keep transactions **short**: do the user interaction and slow calls outside the transaction.
- Lock what you need up front (**SELECT ... FOR UPDATE**) instead of acquiring locks gradually.
- Use **proper indexes**, so updates lock only the rows they need and not large ranges.
- Use a lower isolation level where the business rules allow it.
- Catch the deadlock error and **retry** the transaction.

Q49. How do you defend against SQL injection?

ANSWER

SQL injection happens when user input is glued into a SQL string, so the input can change the meaning of the query. The primary defence is **parameterized queries (prepared statements)**: the SQL text and the values are sent separately, so the database never treats the input as code.

FLOW



EXAMPLE

```
# UNSAFE: input becomes part of the SQL
query = "SELECT * FROM users WHERE name = '" + user_input + "'"
cursor.execute(query)

# SAFE: the value is passed separately
cursor.execute("SELECT * FROM users WHERE name = ?", (user_input,))
```

With the input ' OR '1'='1 against a table of three users, a real test gave:

Tested result

Method	Rows returned
String concatenation	3 (every user, including secrets)
Parameterized query	0

Other layers of defence:

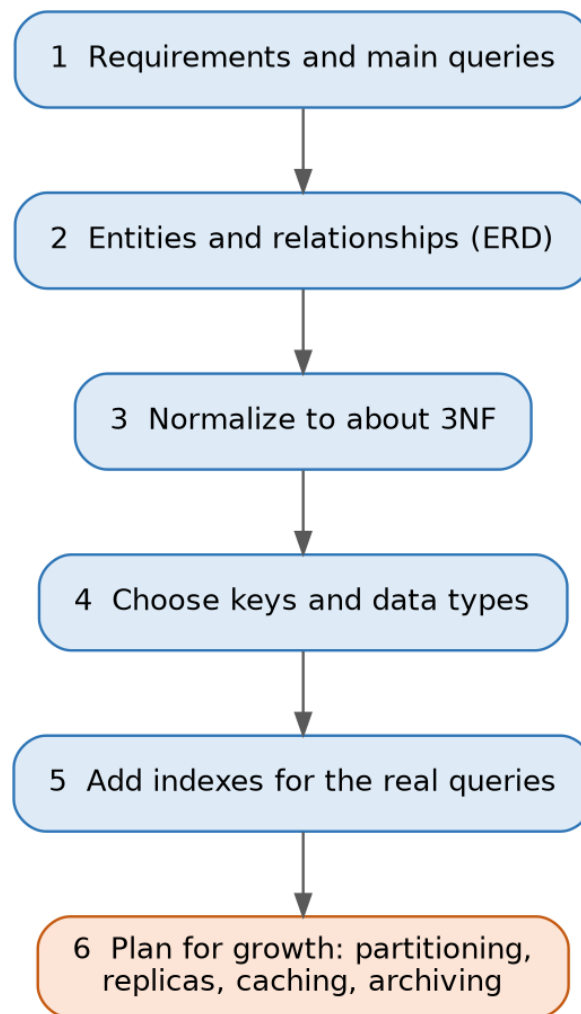
- Use an ORM or query builder correctly, and never concatenate input into raw SQL fragments.
- Give the application account **least privilege** (no DROP, no access to other schemas).
- Placeholders cannot be used for table or column names. For those, check the input against an **allow-list**.
- Do not show raw database errors to users.
- Stored procedures help only if they do not build dynamic SQL from input.

Q50. How would you design a schema that scales?

ANSWER

There is no single right answer. Interviewers want to hear a method: start from the real access patterns, model correctly first, then optimize for the actual bottlenecks.

FLOW



- **Model first.** Normalize to about 3NF so each fact lives in one place. Use clear primary keys (BIGINT identity or UUID) and foreign keys for integrity.
- **Choose data types carefully.** Smaller types mean smaller indexes: use DATE or TIMESTAMP for dates, DECIMAL for money, and avoid storing numbers as text.
- **Index for the queries you really run.** Check the plans, and avoid over-indexing write-heavy tables (Q42).
- **Denormalize selectively** for hot read paths, for example a stored order total or a counter, and keep it consistent.
- **Partition large tables** by time or tenant, so queries and maintenance touch only the relevant partitions, and old data can be archived or dropped cheaply.
- **Scale reads** with read replicas and caching; **scale writes** by batching, sharding, or moving heavy analytics to a warehouse.
- **Avoid anti-patterns:** one giant "key/value" table for everything (EAV), storing comma-separated lists in a column, and SELECT * in hot paths.
- **Plan for change:** migrations, soft deletes or audit tables where the business needs history.

EXAMPLE

```
CREATE TABLE customers (  
  customer_id BIGINT PRIMARY KEY,  
  email       VARCHAR(255) NOT NULL UNIQUE,  
  created_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE orders (  
  order_id    BIGINT PRIMARY KEY,  
  customer_id BIGINT      NOT NULL REFERENCES customers(customer_id),  
  order_date  DATE        NOT NULL,  
  total       DECIMAL(12,2) NOT NULL  
);  
  
-- Supports "recent orders for a customer" without scanning the whole table  
CREATE INDEX idx_orders_customer_date ON orders (customer_id, order_date);
```

Final Notes

WHAT INTERVIEWERS CARE ABOUT IN 2026

According to one 2026 guide, interviewers care less about memorizing syntax and more about problem-solving: translating a business question into a working query, optimizing a slow query, and designing a schema that scales. A typical SQL technical round runs 45 to 60 minutes.

HOW TO PREPARE

1. Master the pattern cheat sheet. If you can write each pattern from memory in your preferred dialect, you cover most live-coding rounds.
2. Practise on LeetCode (Database), HackerRank SQL, DataLemur, StrataScratch and Mode's SQL tutorial.
3. Say out loud why you chose `DENSE_RANK` over `RANK`, or `LEFT JOIN` over `INNER`. Interviewers score the reasoning too.
4. Check which dialect the company uses. Date functions and some syntax differ between PostgreSQL, MySQL and SQL Server.

SOURCES CONSULTED

- Interview Kickstart: SQL Interview Questions and Answers (2026)
- DataCamp: Top 99 SQL Interview Questions and Answers for 2026
- CodeForGeek: Top 50 Essential SQL Interview Questions and Answers (2026)
- Career SQL (AI2SQL): SQL Interview Questions 2026 with Runnable Answers
- Hying: Top 65 SQL Interview Questions and Answers (2026)
- techinterview.org: The SQL patterns behind almost every interview question